Árvores B

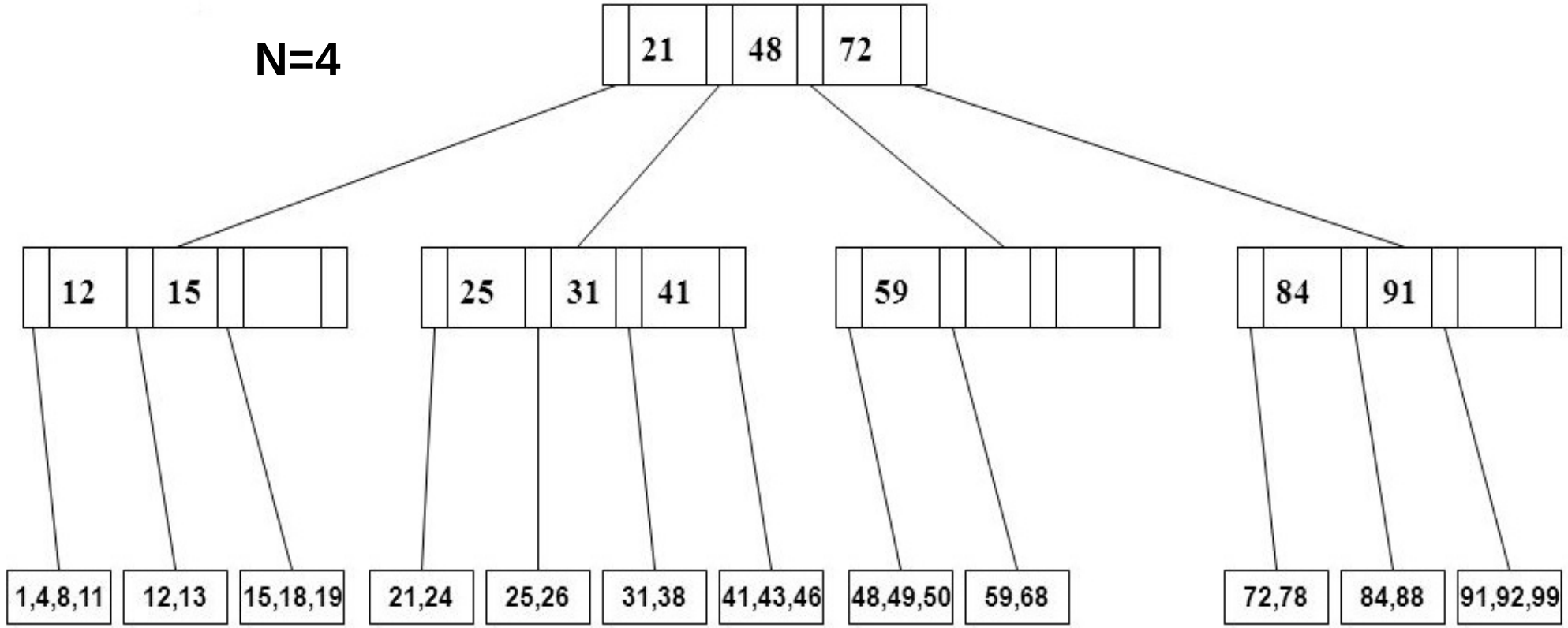
Amanda Ozava Fernandes
Engenharia de Computação
Classificação e Pesquisa de Dados (Samuel Basílio)

Árvores B

- Criadas por Bayes e McCreight em 1972.
- Utiliza mais de uma chave em cada nó da estrutura.
- Assegura que as folhas estão sempre no mesmo nível (auto-balanceada).
- São utilizadas como forma de armazenamento em memória secundária, como em disco magnético ou outros dispositivos de armazenamento secundário.
- Inserções e remoções em tempo logarítmico.

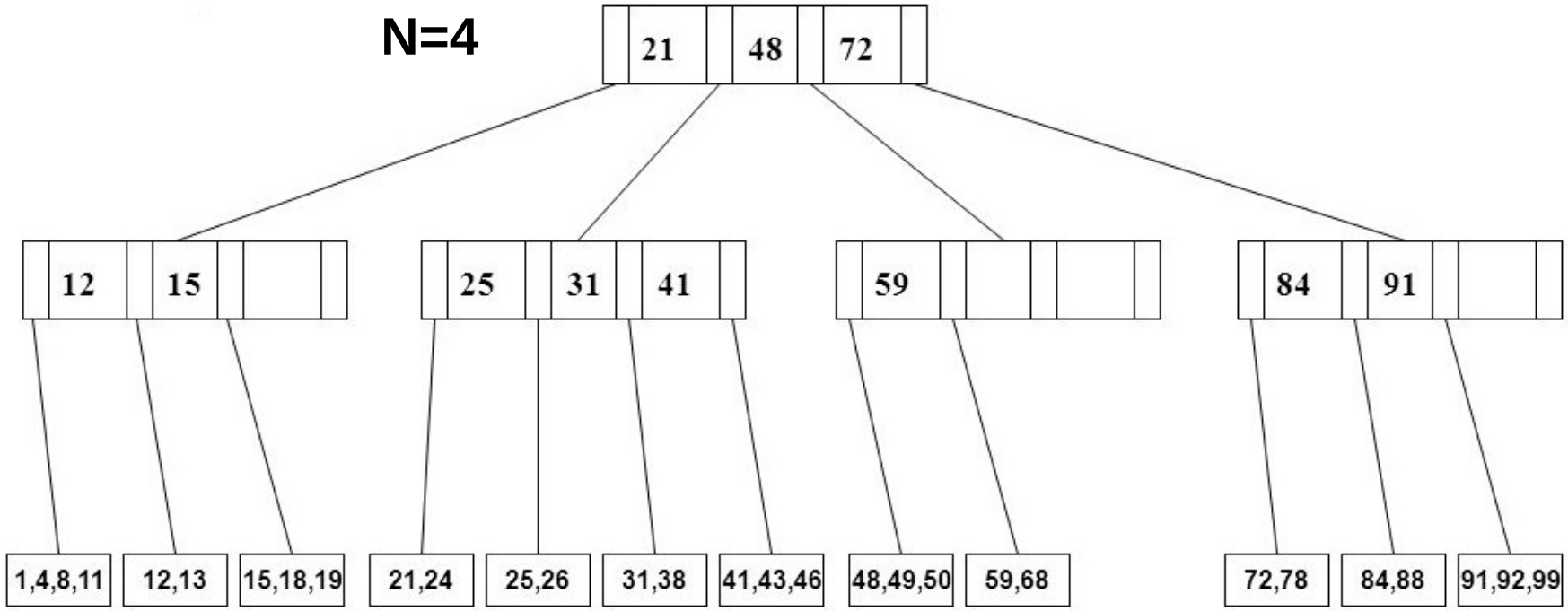
Árvores B – Ordem (N)

- A ordem de uma árvore é a quantidade máxima de subárvores que um nó pode ter. (Knuth, 1978)



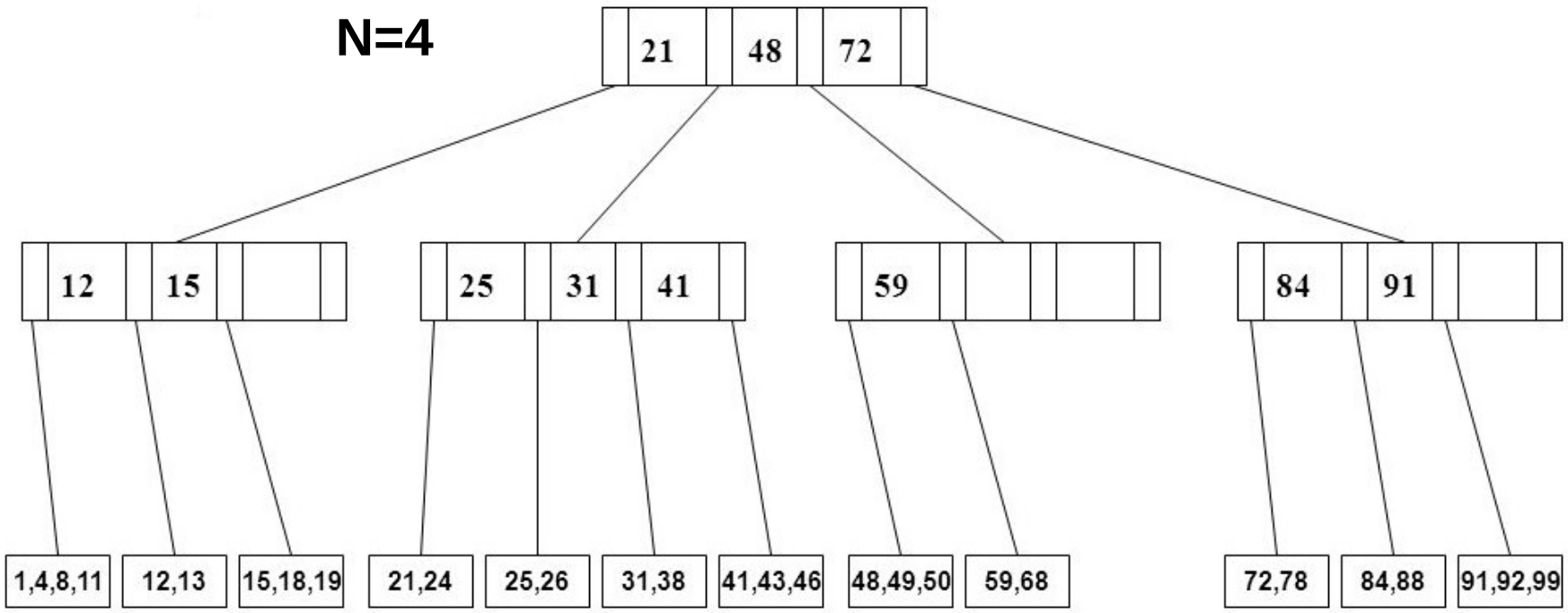
Árvores B – Algumas regras

- A raiz tem no mínimo 2 e no máximo **N** subárvores.



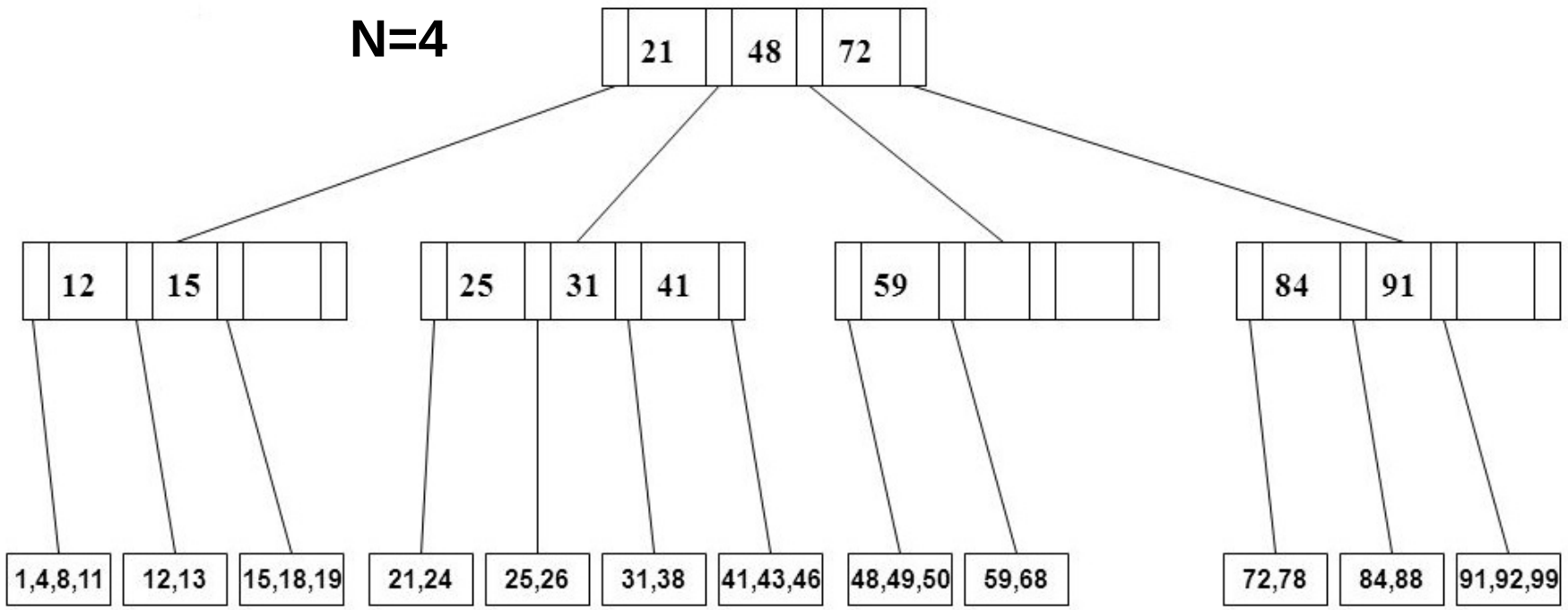
Árvores B – Algumas regras

- Cada um dos nós internos tem entre **⌈N/2⌉ -1** e **N-1** elementos
- Cada um dos nós internos tem entre **⌈N/2⌉** e e **N** subárvores



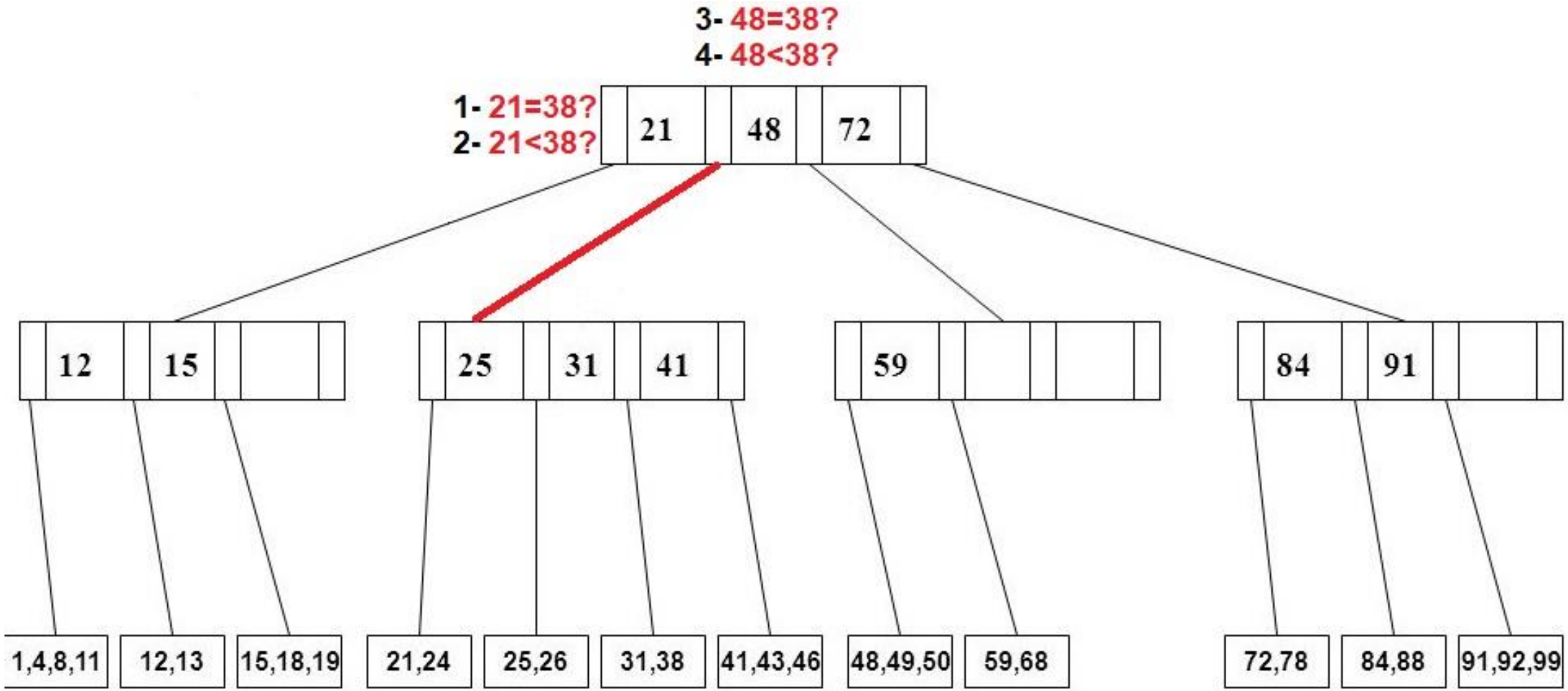
Árvores B – Algumas regras

- As folhas estão sempre no mesmo nível



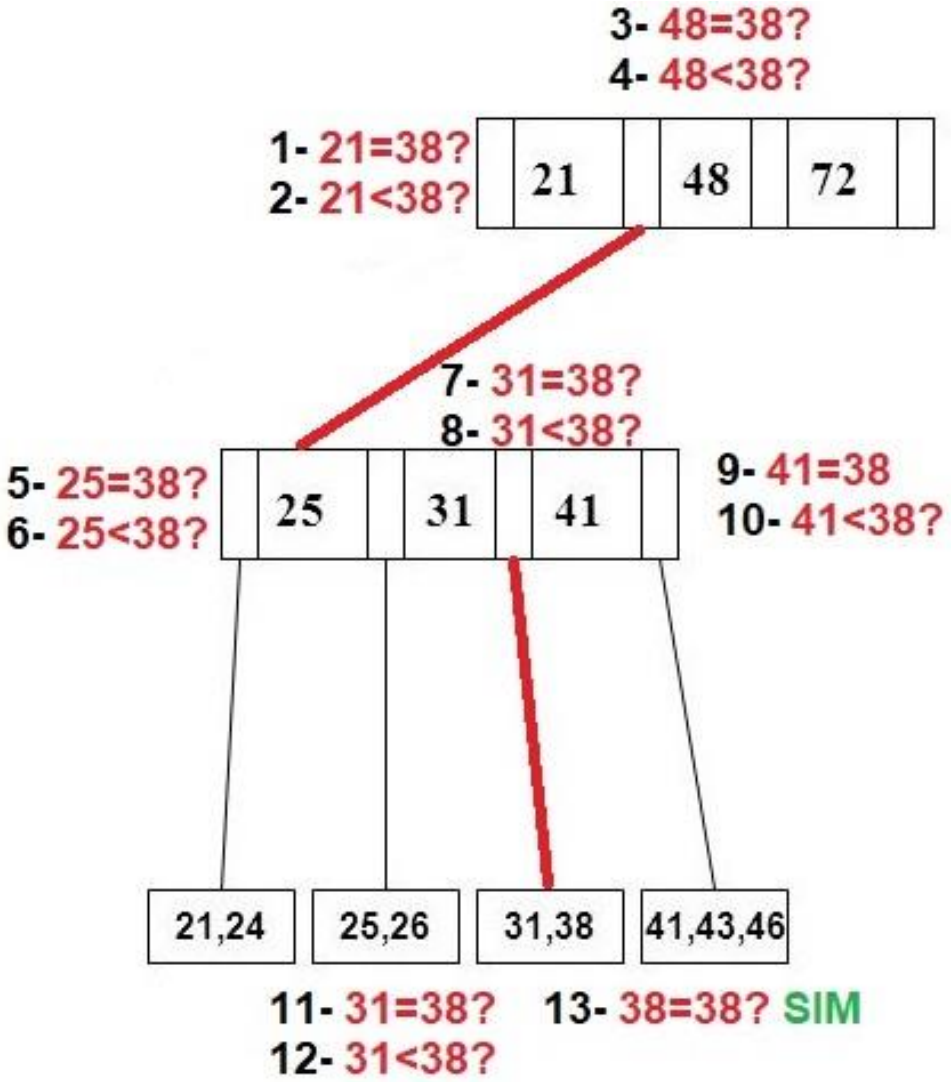
Árvores B – Busca

CHAVE 38



Árvores B – Busca

CHAVE 38



Árvores B – Busca (Pseudocódigo)

```
Busca(k, ponteiroRaiz){
    SE (ponteiroRaiz == NULL ) ENTÃO
        return (chave nao encontrada)
    SENÃO
    //procura k na pagina carregada
      SE (k foi encontrada)
           return (chave encontrada)
       SENÃO
           ponteiro = ponteiro para a próxima página da possível
ocorrência de k
           return (Busca (k, ponteiro))
}
```

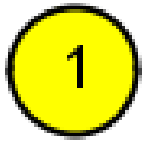
Árvores B – Inserção

- Verificar qual página e posição entrará o elemento.
- Se o elemento couber na página, basta incluí-lo de forma ordenada
- Se não couber o nó deve ser dividido em 2 e o elemento do meio deve ser promovido.

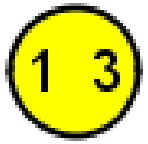
Árvores B – Inserção

- Inserções em uma árvore de ordem N=4

Chave = 1



Chave = 3

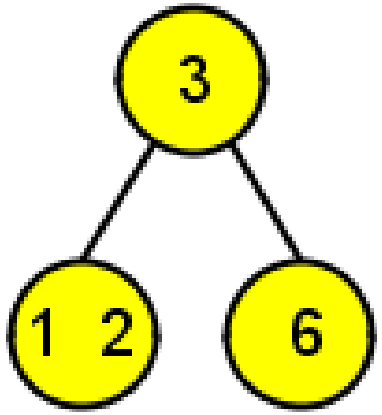


Chave = 2

Árvores B – Inserção

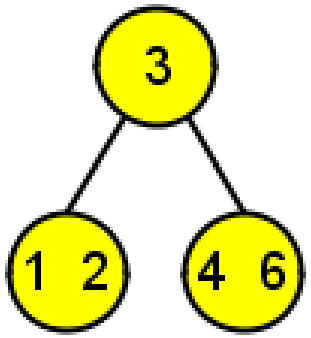
Chave = 6

A inserção do 6 faz com que a árvore que tem ordem N=4, fique com N chaves em um nó, logo, como cada um dos nós internos tem entre **⌈N/2⌉ -1** e **N-1** elementos

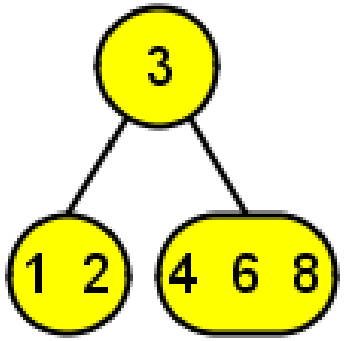


Árvores B – Inserção

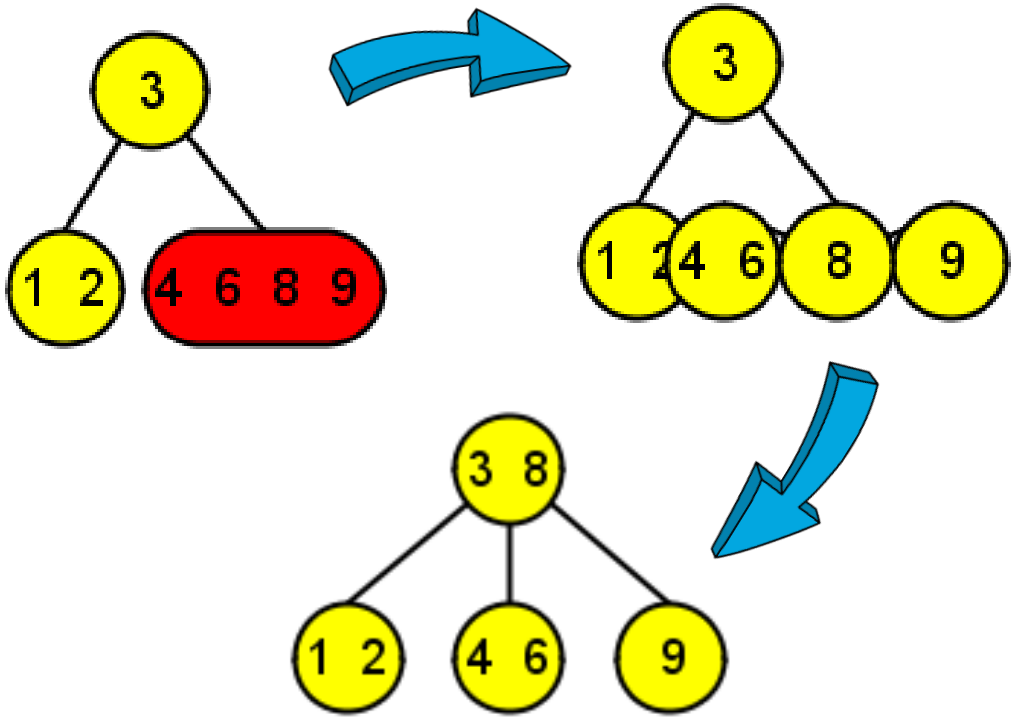
Chave = 4



Chave = 8



Chave = 9



Árvores B- Inserção (PseudoCódigo)

```
insere_B_tharp(nodoB raiz, infoB dado)
   nodoB filho = nulo; /* inicializa p/folha */
   inicio
      SE raiz->folha ENTÃO
         /* insira na folha */
         insere_folha_B(raiz, dado);

      SENÃO
         /* continua descendo */
         filho <- selecionaRamoDescida_B(raiz, dado);
         insere_B(filho, dado);

      FIM SE
      /* retorna ajustando */

      SE nodo_cheio_B(filho) ENTÃO
         /* se nodo possuir 2t+1 chaves, divida,
            lembrando de passar a raiz que se altera */
         divide_nodo_B (raiz, filho);
      FIM SE
   fim
```

Árvores B – Remoção

- Encontrar o elemento a ser removido.

- Existem 4 casos de remoção:

  CASO 1: Remover elemento de uma folha que tem mais que **⌈N/2⌉ - 1**.

  CASO 2: Remover uma chave de um nó não folha.

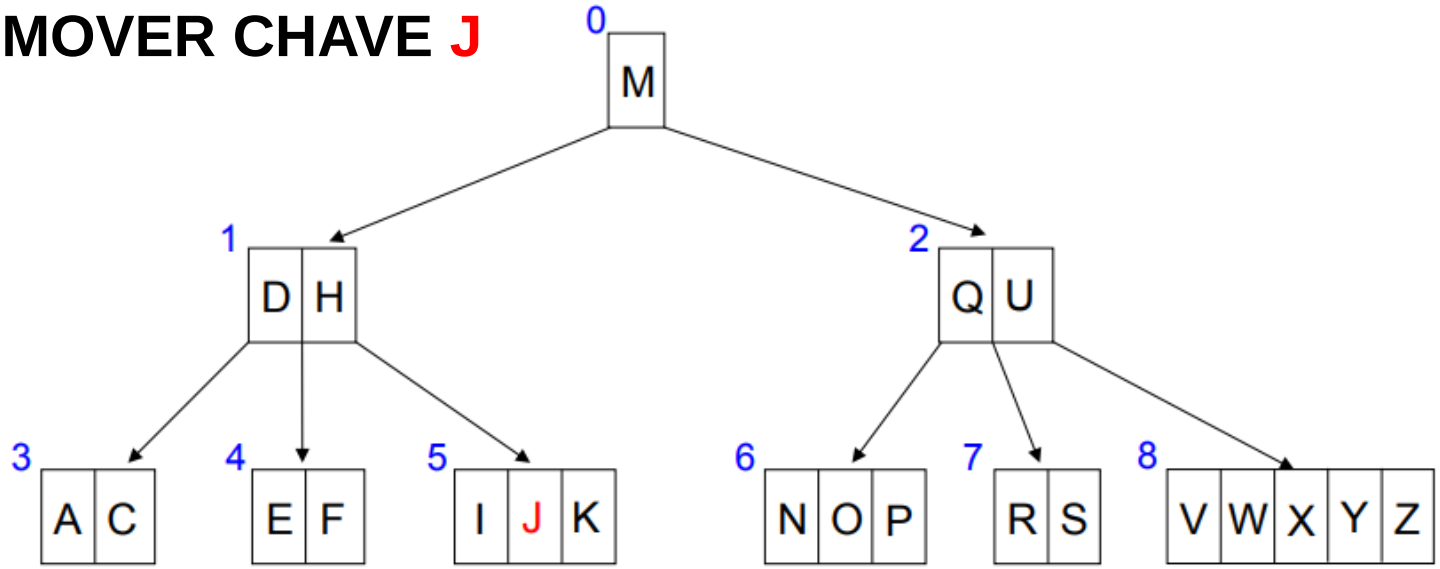
  CASO 3: Remoção de uma chave em um nó, causando *underflow.*

  Caso 4: Remoção de uma chave em um nó, causando *underflow* e a redistribuição não pode ser aplicada.

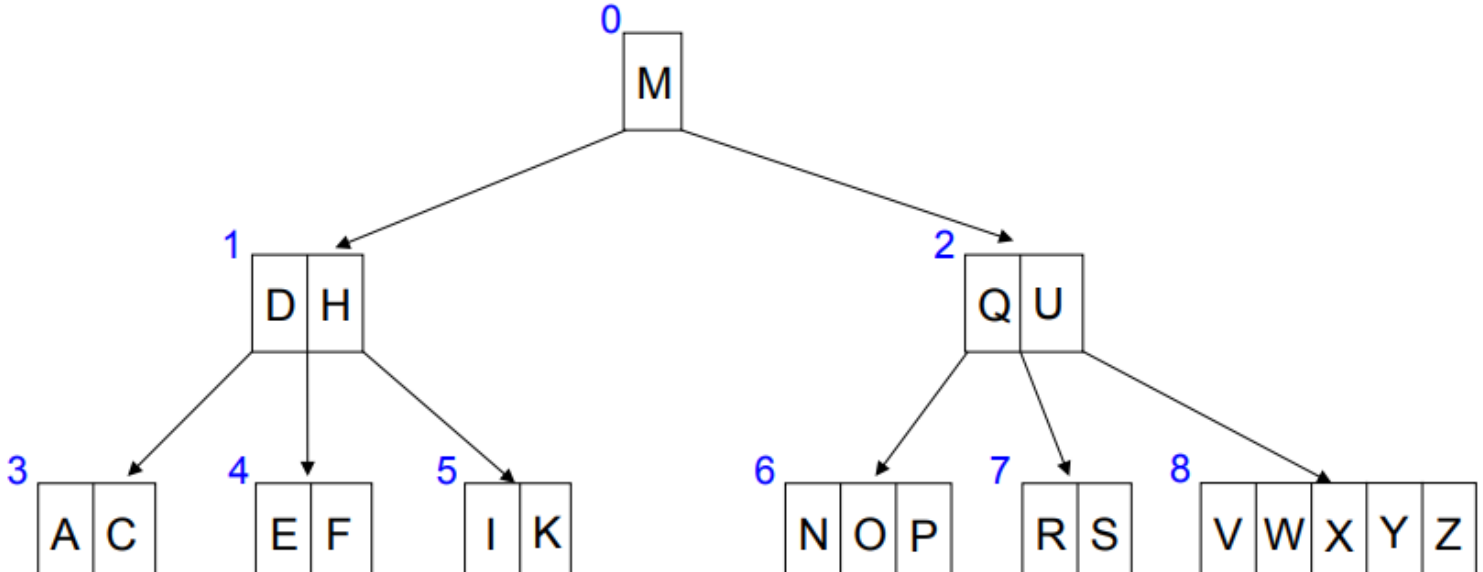
• árvore-B de ordem 6

CASO 1: Remover elemento de uma folha que tem mais que **⌈N/2⌉-1** chaves.

REMOVER CHAVE J



- **⌈N/2⌉-1 = (6/2)-1= 2**

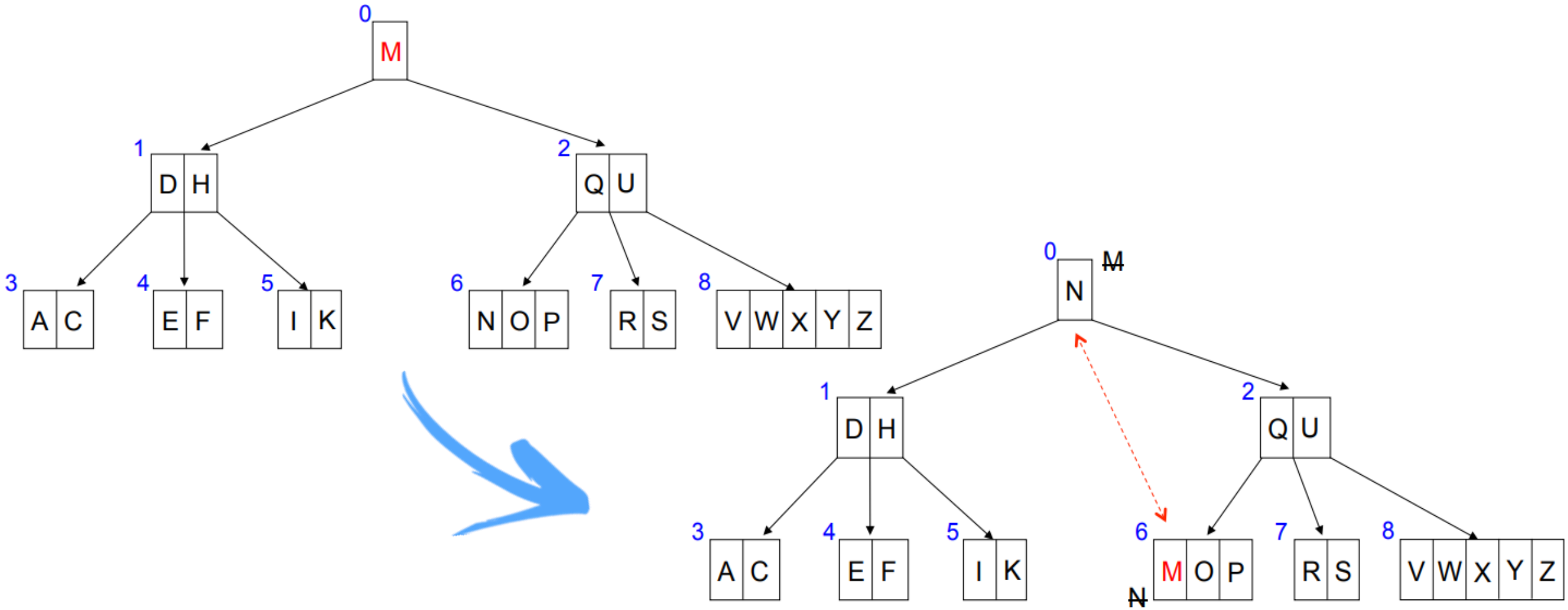


CASO 2: Remover uma chave de um nó não folha

• árvore-B de ordem 6

Passos:
Trocar a chave a ser removida com sua chave sucessora (folha)
Remover a chave diretamente do nó folha

REMOVER CHAVE M



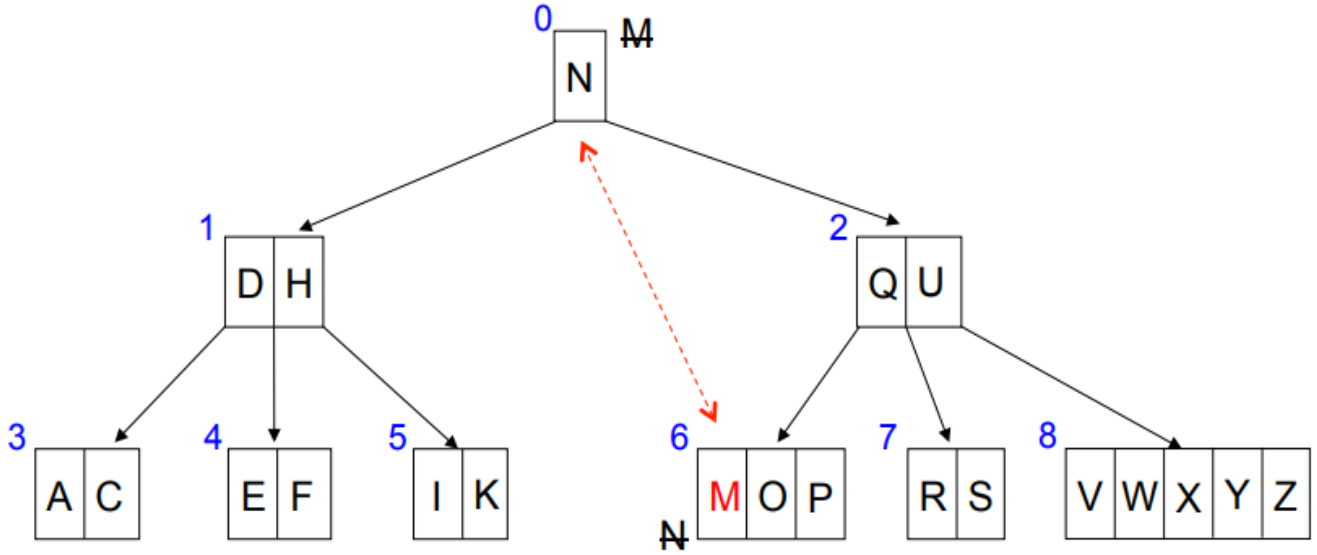
CASO 2: Remover uma chave de um nó não folha

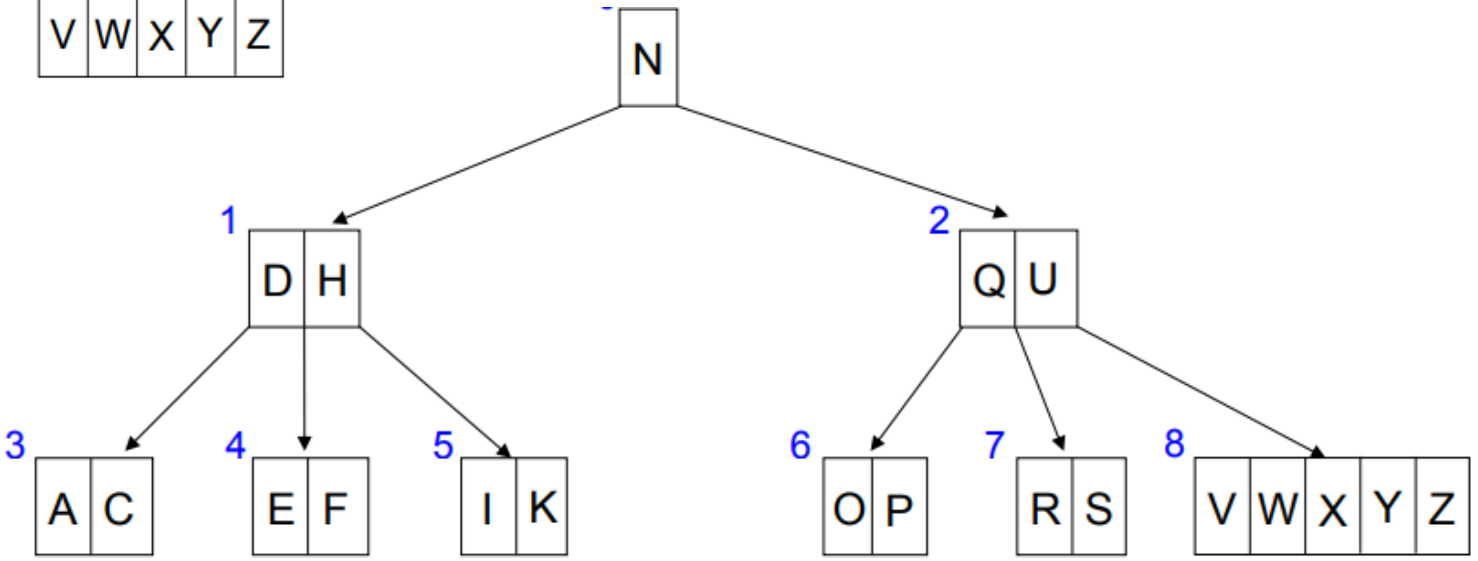
• árvore-B de ordem 6

Passos:
Trocar a chave a ser removida com sua chave sucessora (folha)
Remover a chave diretamente do nó folha

REMOVER CHAVE M

- **⌈N/2⌉-1 = (6/2)-1= 2**



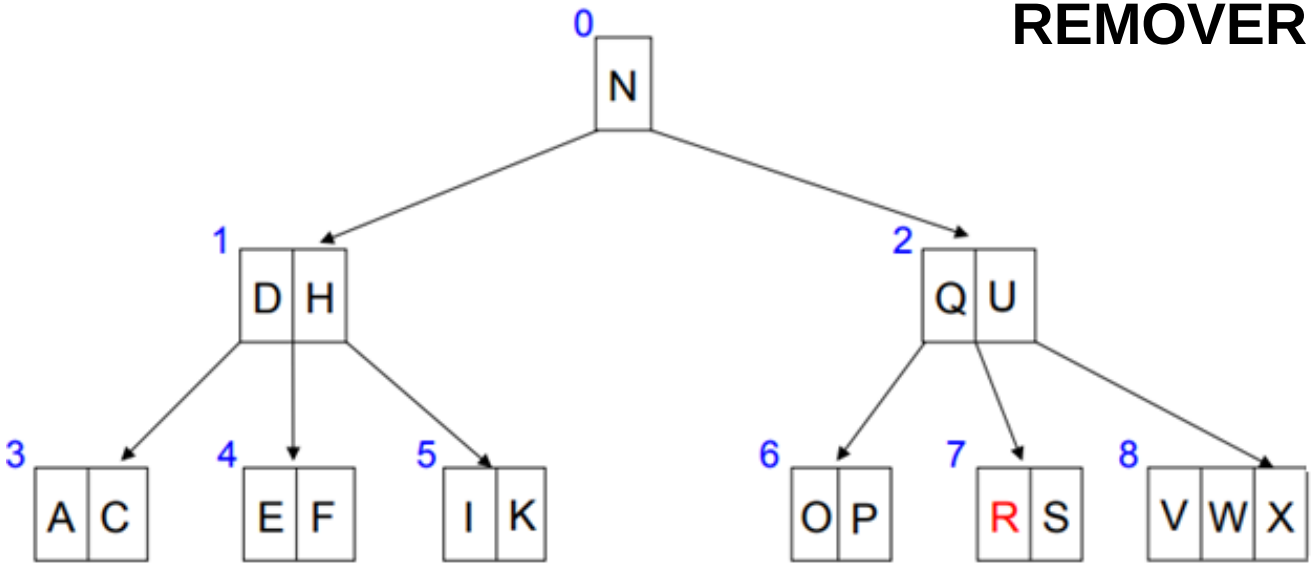


CASO 3: Remover uma chave de um nó causando underflow

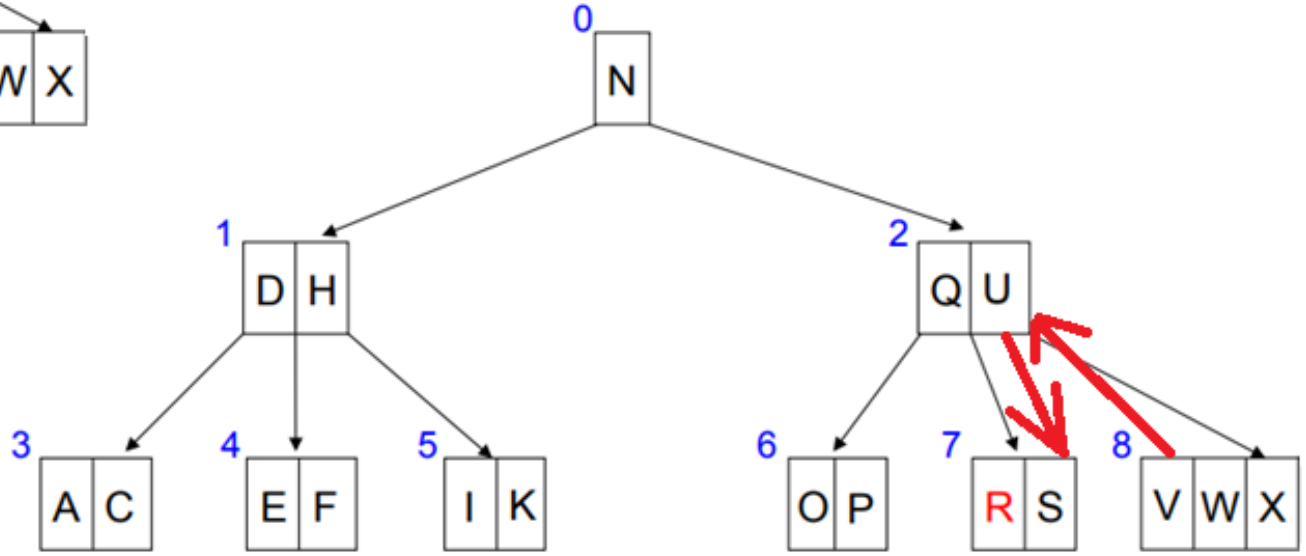
• árvore-B de ordem 6

Passos:
Procurar um nó irmão que tenha quantidade de chaves maior que **⌈N/2⌉ -1**
Se encontrar: redistribuir as chaves entre as páginas e reacomodar a chave separadora, modificando o conteúdo do nó pai

REMOVER CHAVE R



- **⌈N/2⌉-1 = (6/2)-1= 2**

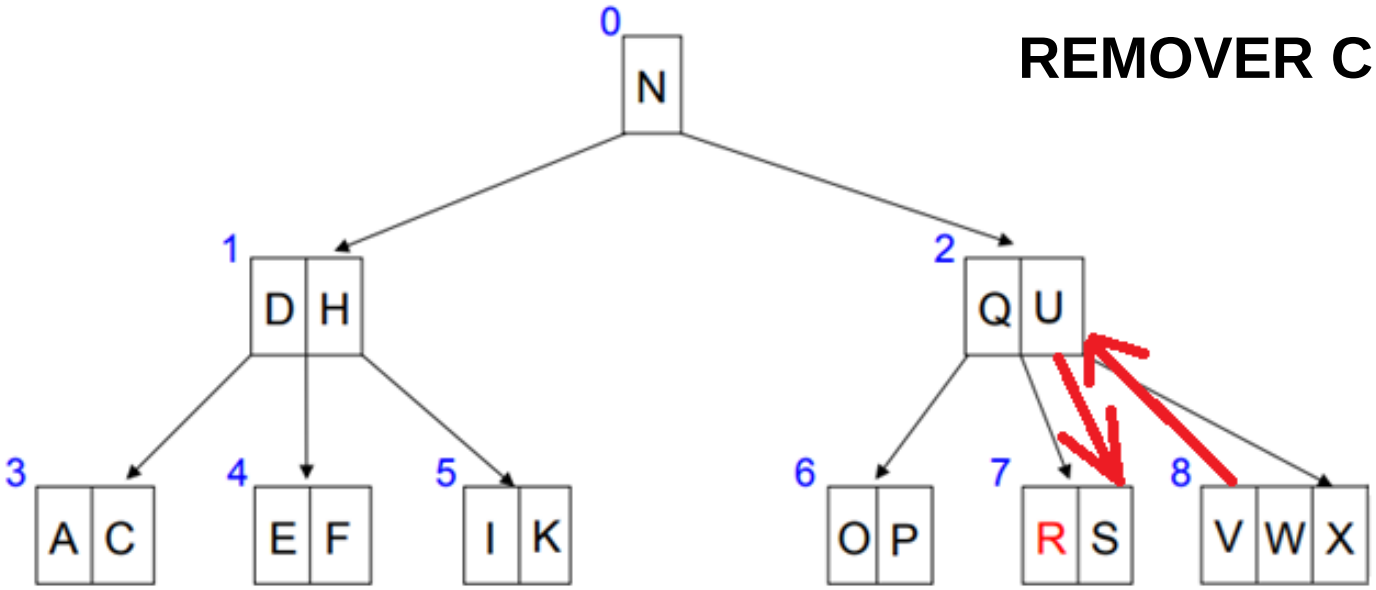


CASO 3: Remover uma chave de um nó causando underflow

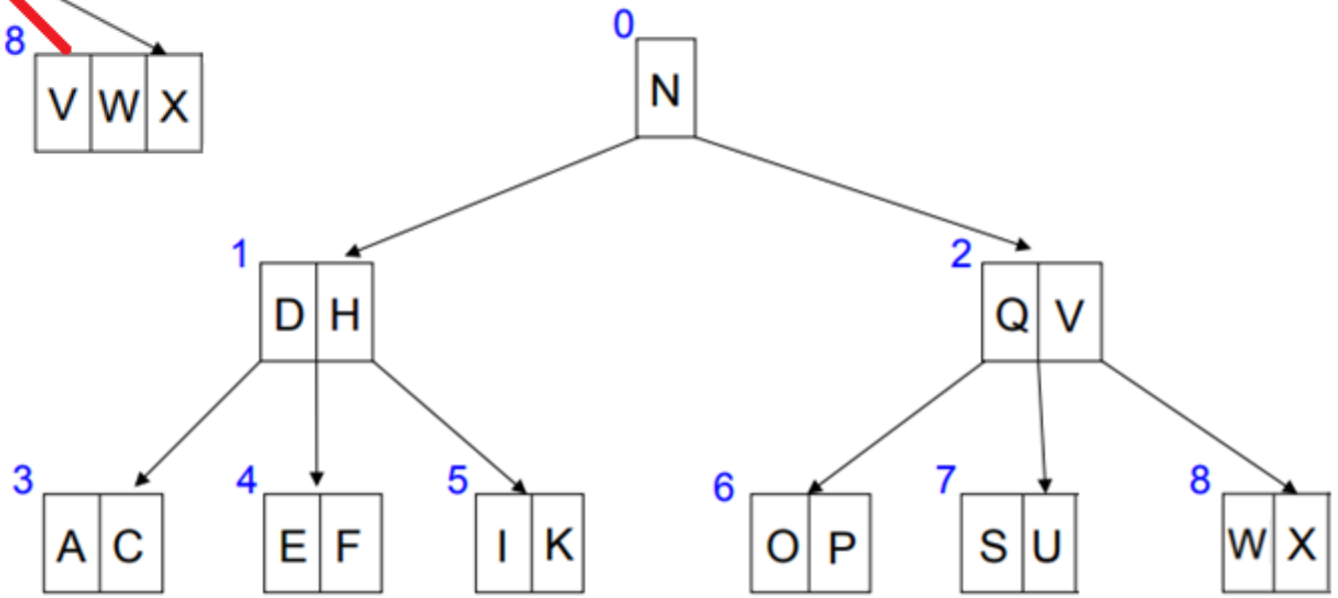
• árvore-B de ordem 6

Passos:
Procurar um nó irmão que tenha quantidade de chaves maior que **⌈N/2⌉ -1**
Se encontrar: redistribuir as chaves entre as páginas e reacomodar a chave separadora, modificando o conteúdo do nó pai

REMOVER CHAVE R



- **⌈N/2⌉-1 = (6/2)-1= 2**

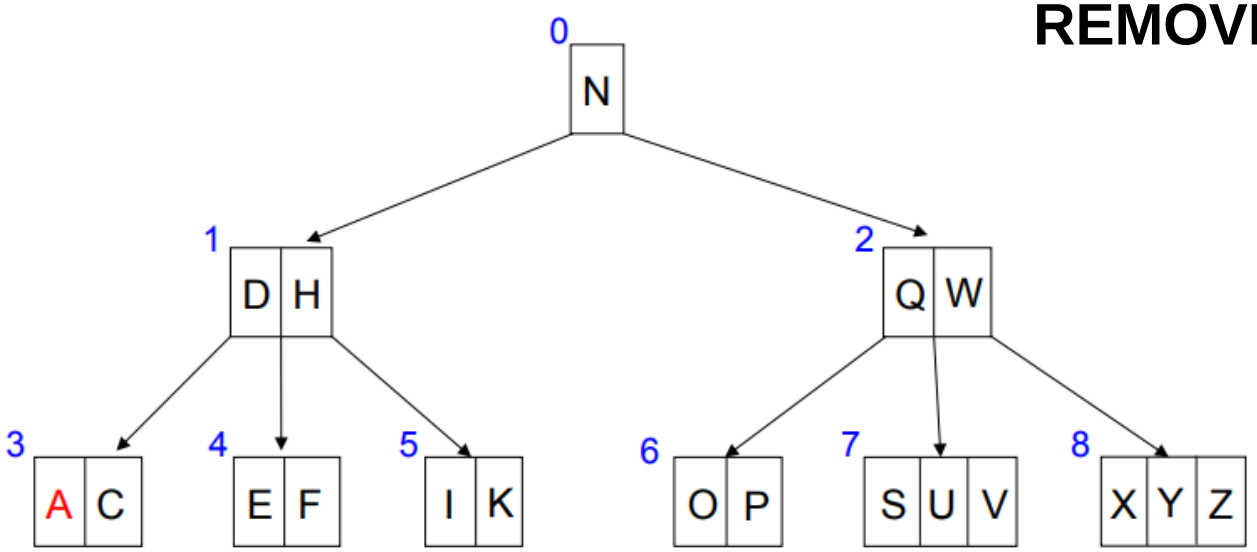


• árvore-B de ordem 6

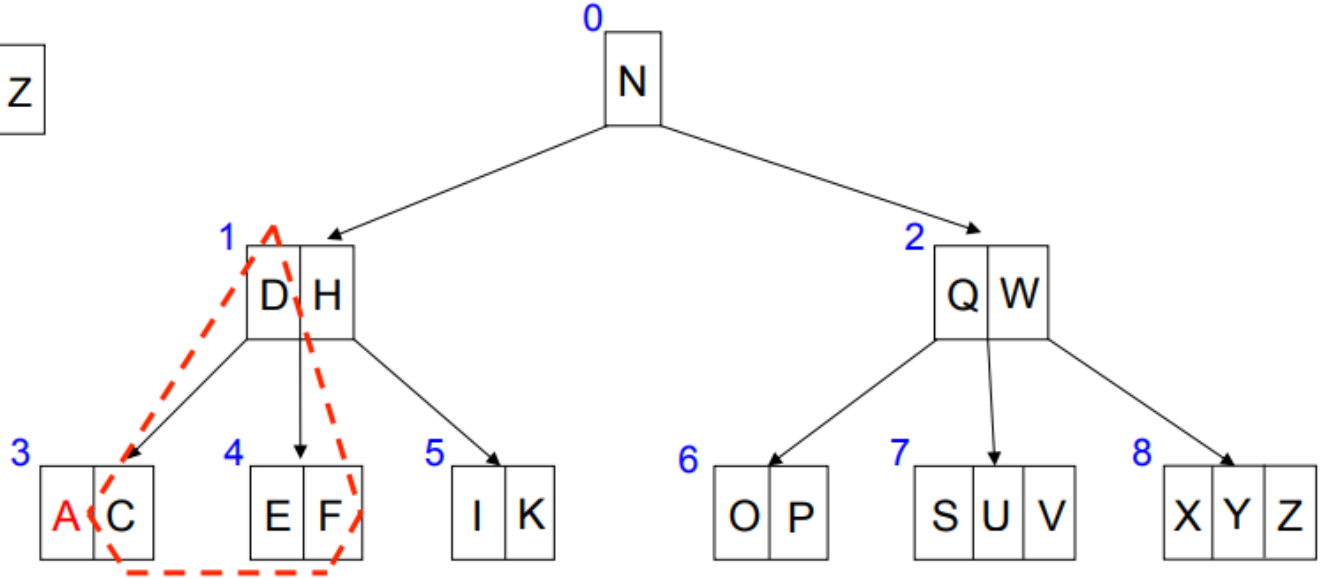
CASO 4: Remover uma chave de um nó causando underflow e o **CASO 3** não é aplicável

Passos:
Realizar a concatenação de dois nós

REMOVER CHAVE A



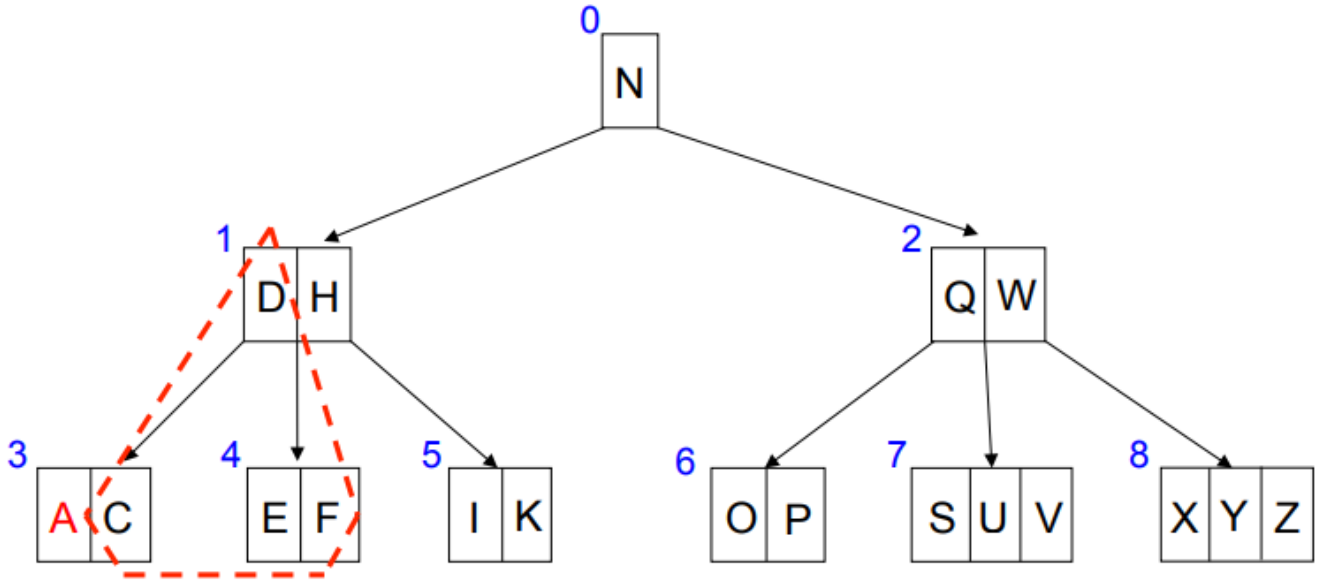
- **⌈N/2⌉-1 = (6/2)-1= 2**

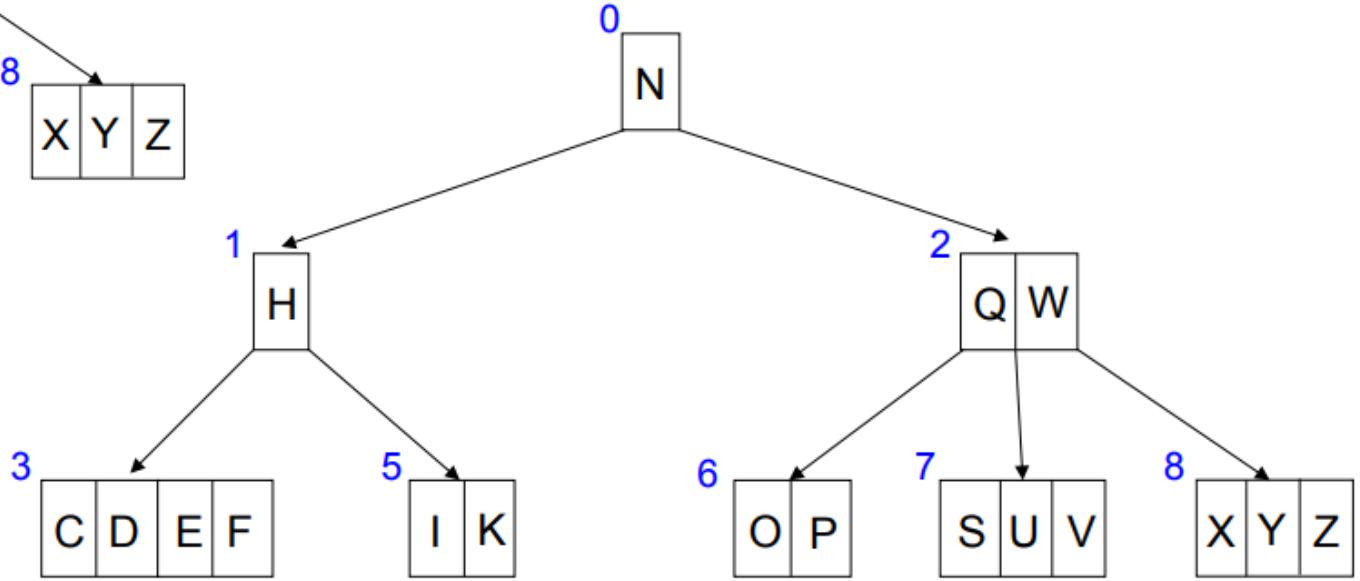


CASO 4: Remover uma chave de um nó causando underflow e o **CASO 3** não é aplicável

Passos: Realizar a concatenação de dois nós

REMOVER CHAVE A



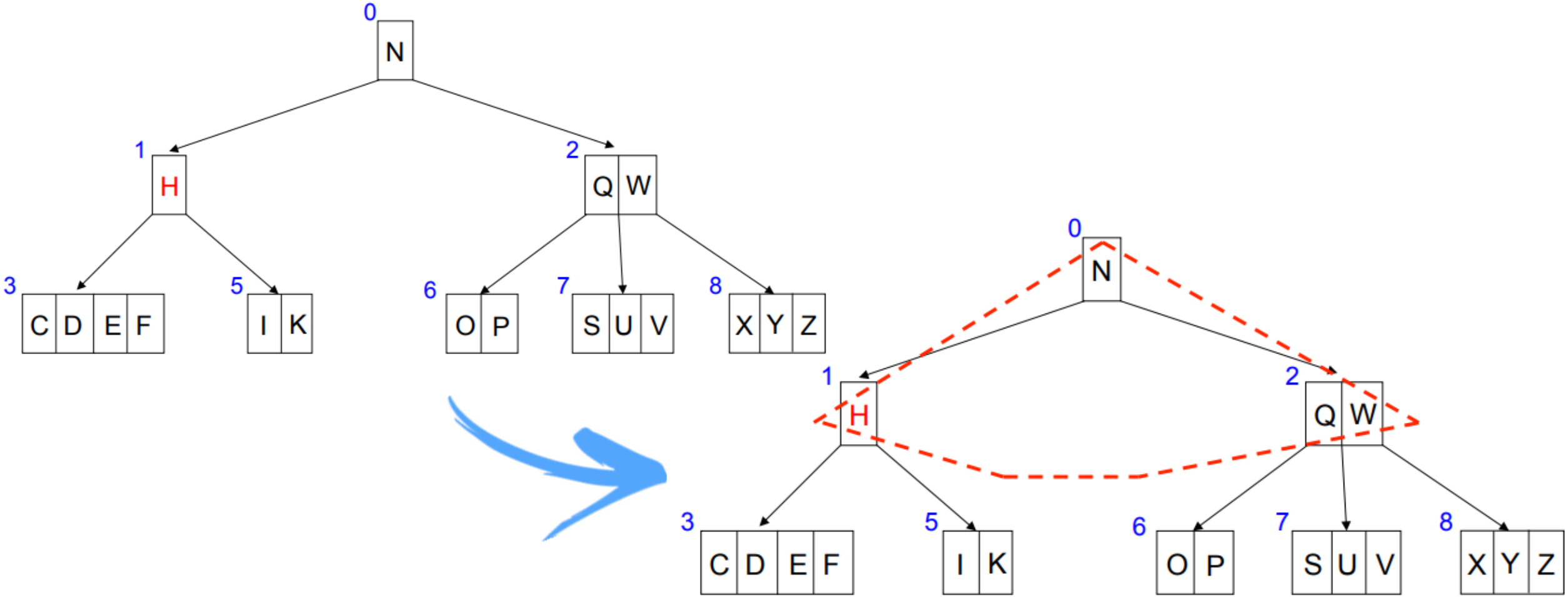


• árvore-B de ordem 6

CASO 4: Remover uma chave de um nó causando underflow e o **CASO 3** não é aplicável

Passos: Realizar a concatenação de dois nós

TIVEMOS UNDERFLOW NA CHAVE H

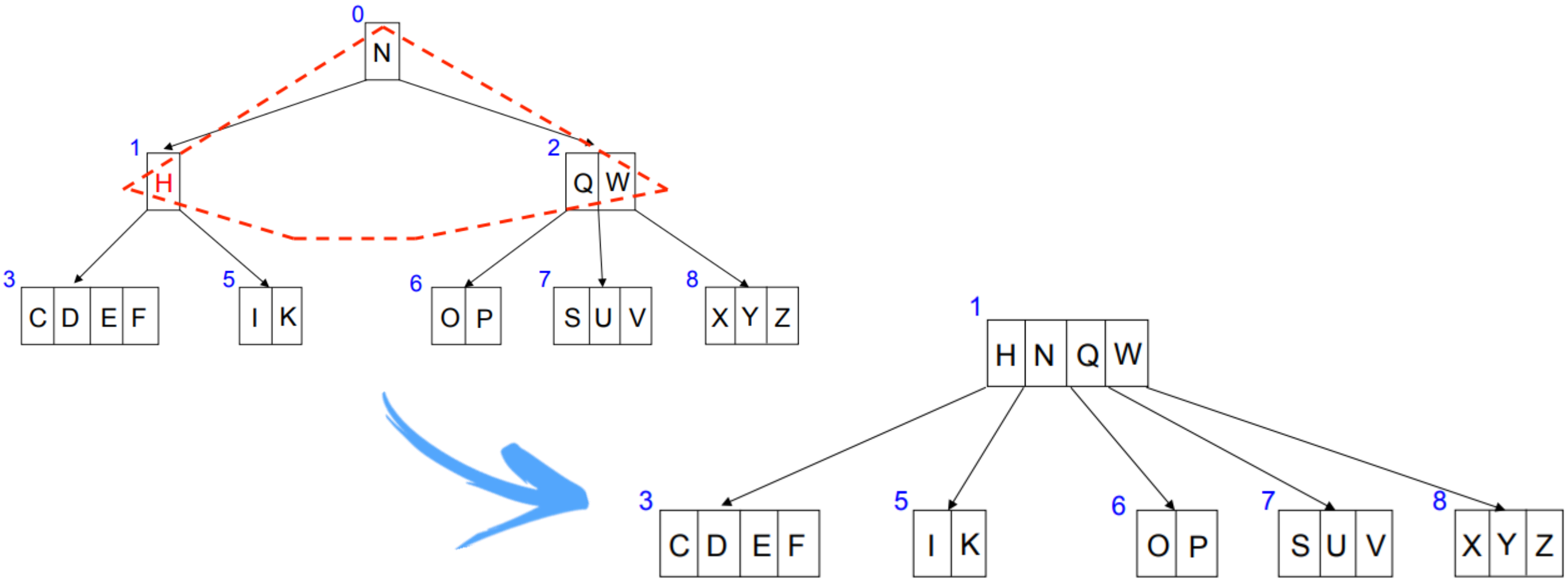


• árvore-B de ordem 6

CASO 4: Remover uma chave de um nó causando underflow e o **CASO 3** não é aplicável

Passos: Realizar a concatenação de dois nós

TIVEMOS UNDERFLOW NA CHAVE H



Árvores B – Remoção (Pseudocódigo)

```
deleta_B(nodoB pai, nodoB nodo, infoB dado)
  variaveis
    nodoB filho;
  inicio
    SE contem_chave_B(nodo, dado) ENTÃO
      SE raiz->folha ENTÃO /* caso 1 */
        deleta_folha_B(nodo, dado);

      SENÃO SE /* caso 2 */
         deleta_interno_B(pai, nodo, dado);
      SENÃO /* caso 3 ou 4*/
        deleta_B(nodo, filho, dado);
         ajusta_nodo_B_tharp(nodo, filho, dado);
      FIM SE

    SENÃO /* desce recursivamente */
      filho <- selecionaRamoDescida_B(nodo, dado);
         /* Chama novamente o deleta_B recursivamente*/
    FIM SE
  fim
```

Dúvidas

Árvores B – Remoção – Pseudocódigo

- CIFERRI, Cristina; PARDO, Thiago. Árvores-B: Remoção. 32 slides. Disponível em: <http://wiki.icmc.usp.br/images/b/bf/SCC0215012014p5arvoreBremocao.pdf> . Acesso em: 12.out. 2019.
- USP Árvores B (B-trees). Disponível em: https://www.ime.usp.br/~pf/estruturas-de-dados/aulas/B-trees.html Acesso em: 12.out.2019.
- CIFERRI Cristina. Árvore B, B* e B+. 157 slides. Disponível em: <http://wiki.icmc.usp.br/images/8/8e/SCC578920131-B.pdf> Acesso em: 12.out.2019.